TP DATA CORE Streaming – Jours 2 à 5 : la boutique Olist, des données au temps réel

Avant de commencer

Vous construisez de zéro la boutique en ligne d'Olist et sa chaîne de données : des données nettoyées pour comprendre les clients par clustering, une boutique avec comptes clients et compte administrateur, une orchestration Airflow, puis un dashboard administrateur alimenté en temps réel par Kafka et Flink. Le seul élément fourni est la base `client_geo`. Tout le reste, vous l'écrivez.

Le contexte

Olist est une place de marché brésilienne qui relie des milliers de petits vendeurs aux grands sites de e-commerce. Elle a publié sur Kaggle environ 99 441 commandes passées entre septembre 2016 et octobre 2018 : Brazilian E-Commerce Public Dataset by Olist. Ces données sont réelles, donc imparfaites : valeurs manquantes, dates incohérentes, villes mal orthographiées, catégories non traduites.

La direction d'Olist veut deux choses. D'abord **comprendre ses clients** : qui achète quoi, combien, à quelle fréquence, avec quelle satisfaction, et regrouper les clients qui se ressemblent. Ensuite **piloter la boutique** depuis un dashboard administrateur, avec certaines informations à jour en continu.

Ce que vous construisez

1. **Demi-journée 1, les données.** Vous auditez et nettoyez le jeu Olist, vous décidez quelles données garder, vous calculez des indicateurs par client et vous faites un premier clustering.
2. **Demi-journée 2, la boutique.** Vous développez une boutique web : inscription et connexion des clients, connexion administrateur, catalogue, panier, commande. Chaque commande crée un nouveau fichier CSV. L'administrateur dispose d'un dashboard.
3. **Demi-journée 3, l'orchestration.** Airflow ramasse les nouveaux CSV, les nettoie avec vos règles, les charge en base, recalcule les indicateurs et les segments de clients, et surveille la qualité.
4. **Journée 4, le temps réel.** Vous installez Kafka et Flink, vous décidez quelles données du dashboard doivent être en temps réel et lesquelles peuvent rester en batch, vous le mettez en œuvre, vous mesurez, puis vous soutenez vos choix.

Organisation

Séquence	Durée	Thème	Lien avec le module (RNCP)
Demi-journée 1	5 h	Audit, nettoyage, données finales, indicateurs clients, clustering	Préparation des données (compétence C29)
Demi-journée 2	5 h	Suite demi-journée 1 + Boutique web : comptes, commandes, CSV, dashboard admin	Source de données de l'application
Demi-journée 3	5 h	Orchestration Airflow : ingestion,	Traitement batch, prolonge l'exercice 1 (batch ou temps

Séquence	Durée	Thème	Lien avec le module (RNCP)
		segments, qualité	réel)
Journée 4	4h	Kafka et Flink, choix des données temps réel, mise en œuvre, mesures, soutenance	Chapitres 2 à 5 ; exercices 2 à 5

Kafka et Flink arrivent à la journée 4 : Kafka transporte les événements de la boutique, Flink calcule en continu les indicateurs temps réel du dashboard (exercice 3 du module).

Soutenance (4h)

Préparation oral (1h30) et ensuite,

Chaque groupe dispose de 30 minutes :

1. **Démonstration** (5 minutes) : un client s'inscrit et commande ; la commande apparaît en direct sur le dashboard administrateur ; quelques minutes plus tard, Airflow l'a intégrée et le client a un segment.
2. **PTT avec Présentation, Introduction, Sommaire, Conclusion, Numéro de slide**

Ensuite je poserai des questions, non comprises dans les 30 minutes.

Ce qui vous est fourni

Un seul fichier : `client_geo.sql`. Il crée et remplit la table `client_geo` de PostgreSQL : une géolocalisation pour chacun des 99 441 `customer_id` du fichier `olist_customers_dataset.csv`.

Colonne	Type	Contenu
`customer_id`	VARCHAR(32), clé primaire	Identifiant du client pour une commande, même valeur que dans les fichiers Olist
`latitude`	NUMERIC(9,6)	Latitude du client
`longitude`	NUMERIC(9,6)	Longitude du client

Les règles du jeu

- Vous êtes 9, répartis en **3 groupes de 3**. Chaque groupe construit sa propre application complète, dans son propre dépôt Git. Chaque séquence se termine par un commit.
- Aucun code n'est fourni. Le document donne des spécifications, des contraintes et des indications.
- Chaque groupe tient un **carnet** (`CARNET.md` dans le dépôt) : décisions prises et leur justification, mesures chiffrées. Les soutenances s'appuient dessus.
- La documentation officielle est votre première ressource : pandas, scikit-learn, FastAPI, Airflow, Apache Kafka, Apache Flink, confluent-kafka pour Python, PostgreSQL, DBeaver.

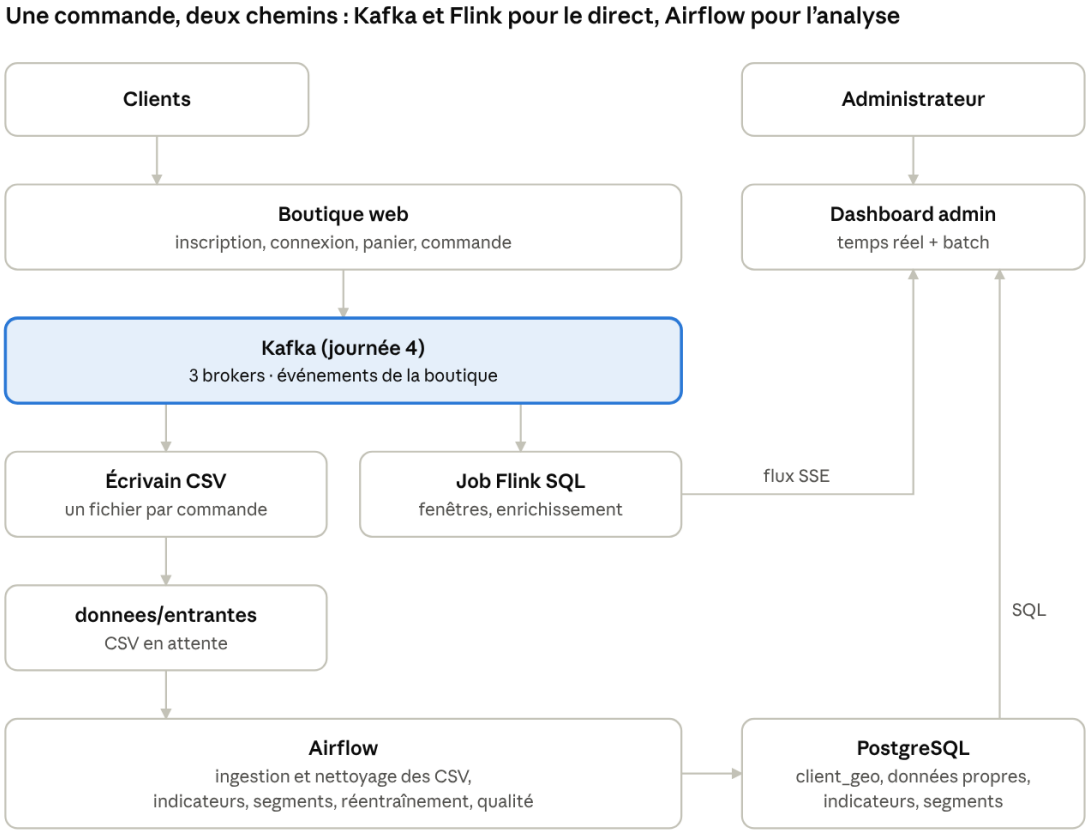
La pile technique imposée

Composant	Image ou outil	Rôle
PostgreSQL	`postgres:16` ou plus récent	Base `olist` (dont `client_geo`) et base des métadonnées d'Airflow
Apache Airflow	`apache/airflow:3.x` (mode `standalone`)	Orchestration des traitements batch et des tâches d'exploitation
Apache Kafka	`apache/kafka` (mode KRaft, sans ZooKeeper), 3 brokers	Journal des événements
Kafka UI	`kafbat/kafka-ui`	Interface web pour observer le cluster
Python 3.11 ou plus	`confluent-kafka`, `psycopg2-binary` ou `psycopg`, `pandas`, `folium`	Simulateur, consommateurs, notebooks
JupyterLab	à construire vous-même (image Python + `pip install`)	Votre poste de travail dans le réseau Docker
Apache Flink	flink (JobManager et TaskManager), avec les connecteurs Kafka et JDBC	Calcul continu des indicateurs temps réel (journée 4)

S'y ajoutent les bibliothèques Python de votre application : `pandas` pour le nettoyage, `scikit-learn` et `joblib` pour le clustering, `FastAPI` et `uvicorn` pour la boutique, `folium` ou `plotly` pour les cartes et graphiques du dashboard. Pour explorer la base, installez sur votre poste le client graphique gratuit DBeaver Community.

Rappel du jour 1 : depuis votre navigateur on écrit `localhost:port` ; depuis un programme qui tourne dans un conteneur, on écrit `nom_du_service:port`.

L'architecture que vous allez construire



architecture cible · à construire en 4 séquences

La boutique publie ses événements ; Airflow transforme les commandes en données propres et en segments de clients ; Kafka transporte les événements et Flink calcule la partie temps réel du dashboard. Jusqu'à la journée 4, la boutique écrit directement ses fichiers CSV ; Kafka s'intercale ensuite entre les deux.

Demi-journée 1 (5 h) : comprendre, nettoyer, segmenter

Objectif : transformer les 9 fichiers bruts d'Olist en un modèle de données propre dans PostgreSQL, en décidant vous-mêmes ce que vous gardez, puis en tirer une première segmentation des clients. Vos règles de nettoyage serviront à nouveau quand Airflow traitera les commandes de votre boutique : écrivez-les pour être réutilisées.

Etapes :
Étape 1 : l'environnement (PostgreSQL, JupyterLab) et le chargement de `client_geo`
Étape 2 : comprendre le jeu Olist

Etapes :
Étape 3 : l'audit de qualité
Étape 4 : les règles de nettoyage, en module réutilisable
Étape 5 : choisir les données finales
Étape 6 : les indicateurs par client
Étape 7 : le clustering

Étape 1 : l'environnement

Téléchargez l'archive depuis la page Kaggle (un compte gratuit suffit) et décompressez-la dans un dossier `donnees/olist` de votre dépôt. Ajoutez `donnees/` au fichier `.gitignore`.

Écrivez un `docker-compose.yml` avec deux services pour commencer :

Service	Exigences
`postgres`	Utilisateur, mot de passe et base `olist`. Volume nommé pour les données. Port 5432 ouvert vers votre ordinateur (DBeaver s'y connectera). Contrôle de santé avec `pg_isready`.
`jupyter`	Image construite par vous (un `Dockerfile` : image Python officielle et `pip install` des bibliothèques). Port 8888. Le dépôt monté dans `/workspace`, `donnees/olist` en lecture seule.

Indications :

- Depuis votre ordinateur (DBeaver), PostgreSQL s'appelle `localhost`. Depuis JupyterLab ou un autre conteneur, il s'appelle `postgres`.
- Pour charger `client_geo.sql`, suivez le guide DBeaver ci-dessous.
- Si vous préférez automatiser plus tard : l'image `postgres` exécute une seule fois, au premier démarrage, les fichiers `.sql` placés dans `/docker-entrypoint-initdb.d`. N'utilisez pas les deux méthodes à la fois.

Guide : charger `client_geo.sql` dans PostgreSQL avec DBeaver

1. **Installer DBeaver.** Téléchargez DBeaver Community sur dbeaver.io et installez-le. L'interface existe en français ; les noms anglais des menus sont donnés entre parenthèses.
2. **Démarrer PostgreSQL.** Dans le dossier du dépôt : `docker compose up -d postgres`, puis `docker compose ps` doit afficher le service en `(healthy)`.
3. **Créer la connexion.** Menu Base de données › Nouvelle connexion (Database › New Database Connection), choisissez PostgreSQL, puis renseignez : hôte `localhost`, port `5432`, base `olist`, et l'utilisateur et le mot de passe de votre docker-compose. Cochez « Enregistrer le mot de passe ».

4. **Installer le pilote.** Cliquez sur Tester la connexion (Test Connection). La première fois, DBeaver propose de télécharger le pilote PostgreSQL : acceptez. Le test doit répondre « Connecté ». Terminez.
5. **Ouvrir le script.** Menu Fichier › Ouvrir un fichier (File › Open File), choisissez `client_geo.sql`. Le fichier fait environ 6 Mo.
6. **Associer le script à la base.** En haut de l'éditeur, le sélecteur de source de données active doit indiquer votre connexion et la base `olist` (sinon, cliquez dessus ou utilisez Ctrl+9 pour la choisir).
7. **Exécuter tout le script.** Utilisez Exécuter le script SQL (Execute SQL Script) avec **Alt+X**, et non Ctrl+Entrée qui n'exécute que l'instruction sous le curseur. Le script crée la table (3 colonnes) puis insère 99 441 lignes par paquets de 1 000. Suivez l'avancement dans l'onglet des statistiques ; comptez moins d'une minute.
8. **Vérifier.** Dans le navigateur de bases à gauche, rafraîchissez (F5) olist › Schémas › public › Tables : `client_geo` apparaît. Ouvrez un nouvel éditeur SQL (Ctrl+]) et exécutez `SELECT count(*) FROM client_geo;` : le résultat doit être 99 441. Double-cliquez sur la table pour voir ses trois colonnes et sa clé primaire (onglet Propriétés) et ses données (onglet Données).

En cas de problème :

- **« Connection refused »** : le conteneur n'est pas démarré, ou le port 5432 n'est pas publié dans le docker-compose, ou un autre PostgreSQL installé sur votre poste occupe déjà ce port (publiez alors le conteneur sur 5433).
- **« la relation « client_geo » existe déjà »** (relation already exists) : le script a déjà été exécuté. Vérifiez d'abord le nombre de lignes ; s'il est faux, exécutez `DROP TABLE client_geo;` puis relancez le script.
- **DBeaver très lent ou à court de mémoire** : chargez le fichier en ligne de commande, sans l'ouvrir : `docker compose exec -T postgres psql -U <utilisateur> -d olist < client_geo.sql`. Puis vérifiez dans DBeaver.
- **Accents mal affichés** : le script est en UTF-8 ; vérifiez l'encodage dans Fenêtre › Préférences › Espace de travail (Window › Preferences › Workspace).

Étape 2 : comprendre le jeu Olist

Ouvrez les 9 fichiers dans un notebook avec pandas et donner votre cheminement de nettoyage par écrit dans `CARNET.md`

Indice :

- Comment les tables sont liées entre elles ?
- Quelles informations dans une colonne est importante ?
- Comment garder les informations importantes de ces colonnes ? (Chercher les agrégations en Python)

Étape 3 : l'audit de qualité

Avant de nettoyer, on mesure. Pour chaque fichier, cherchez les problèmes ci-dessous

Famille de problème
Valeurs manquantes
Doublons
Cohérence temporelle
Cohérence entre fichiers
Valeurs aberrantes

Indication : `df.isna().sum()`, `df.duplicated()`, `df.describe()` et `value_counts()` couvrent une grande partie du travail.

Étape 4 : les règles de nettoyage

Pour chaque problème de l'audit, décidez d'une règle et notez-la dans `CARNET.md` avec sa justification. Il n'y a pas de réponse unique, mais chaque choix doit être défendable.

Puis écrivez ces règles dans un module Python `src/nettoyage.py`, et non dans le notebook :

- une fonction par table (`nettoyer_commandes(df)`, `nettoyer_articles(df)`…) qui prend un DataFrame brut et renvoie le DataFrame propre **et** les lignes rejetées avec la raison du rejet ;
- les conversions de types (dates en UTC avec le fuseau `America/Sao_Paulo`, montants en décimal) ;
- aucune lecture ni écriture de fichier dans ces fonctions : elles transforment, c'est tout. C'est ce qui permettra à Airflow de les réutiliser telles quelles ;
- quelques tests avec `pytest` (un petit DataFrame fabriqué à la main par règle).

Vérifiez que lancer deux fois le nettoyage sur des données déjà propres ne change rien.

Étape 5 : choisir les données finales

Toutes les colonnes ne méritent pas d'être gardées. Vos données finales serviront à trois usages : la boutique (catalogue, clients, commandes), le dashboard administrateur et le clustering. Pour vous aider vous pouvez remplir ce tableau pour chaque colonne des fichiers source :

Table source	Colonne	Gardée ?	Sous quelle forme (renommée, convertie, agrégée)	Usage : boutique, dashboard, clustering	Justification

Traduisez vos choix en un schéma SQL (`sql/schema.sql`, clés primaires et étrangères comprises) et chargez les données nettoyées dans PostgreSQL. Conservez aussi les lignes rejetées dans des tables `rejets_<table>`.

Le modèle en étoile du dashboard

Le dashboard n'interroge pas les données comme la boutique. La boutique écrit les commandes une par une ; le dashboard lit et agrège des centaines de milliers de lignes. Pour lui, construisez un **modèle en étoile** : une **table de faits** au centre, entourée de **tables de dimensions**.

Dessinez le schéma en étoile dans `CARNET.md`, écrivez-le dans `sql/etoile.sql` et alimentez-le à partir de vos tables nettoyées. Vérifiez que le chiffre d'affaires total calculé sur `fait_ventes` est identique à celui calculé sur les tables nettoyées : sinon, votre grain ou vos jointures dupliquent des lignes.

Étape 6 : les indicateurs par client

Le clustering se fait à l'échelle de la personne. Construisez une table `indicateurs_clients` avec une ligne par personne. Point de départ classique : le modèle **RFM**.

Famille	Indicateurs possibles
Récence	Jours depuis la dernière commande (par rapport à la date la plus récente du jeu)
Fréquence	Nombre de commandes
Montant	Total dépensé, panier moyen

Étape 7 : le clustering

Avec scikit-learn, segmentez les clients :

1. Choisissez 4 à 8 indicateurs, transformez-les et standardisez-les.
2. Entraînez des `KMeans` pour k de 2 à 10. Choisissez k et justifiez.
3. Décrivez chaque segment par la moyenne de ses indicateurs et donnez-lui un nom parlant.
4. Enregistrez le modèle et sa standardisation avec `joblib` dans `modeles/`, et le segment de chaque client dans une table `segments_clients` (avec la date et la version du modèle).

Demi-journée 2 (5 h) : la boutique en ligne

Objectif : développer la boutique web d'Olist sur vos données propres. Les clients s'inscrivent, se connectent, remplissent un panier et commandent. Chaque commande produit un nouveau fichier CSV. Un administrateur se connecte à un espace réservé et consulte un dashboard.

Contenu
Étape 1 : concevoir les pages et les comptes
Étape 2 : inscription, connexion, catalogue, panier
Étape 3 : la commande et son fichier CSV
Étape 4 : l'espace administrateur et son dashboard

Contenu
Étape 5 : le robot de trafic et les tests de sécurité

Étape 1 : concevoir les pages et les comptes

Sur papier ou au tableau, dessinez les pages et le parcours d'un client, puis celui d'un administrateur. La boutique doit offrir au minimum :

Qui	Pages et fonctions
Visiteur	Catalogue par catégorie, recherche simple, fiche produit (catégorie, prix, photo factice, poids), inscription, connexion
Client connecté	Panier (ajouter, modifier la quantité, retirer), commande avec choix du paiement (carte en 1 à 10 échéances, boleto, bon d'achat), historique de ses commandes, déconnexion
Administrateur	Connexion par une page dédiée, dashboard, liste des clients avec leur segment, liste des dernières commandes, déconnexion

Étape 2 : inscription, connexion, catalogue, panier

Ajoutez au docker-compose un service `boutique` (port 8000) construit par vous, avec **FastAPI** servi par `uvicorn`. Les pages HTML sont produites par des gabarits Jinja (`Jinja2Templates`) ; les formulaires se lisent avec `Form` (bibliothèque `python-multipart`).

Exigences :

- **Mots de passe** : jamais stockés en clair. Hachez-les (bibliothèque `pwdlib` avec Argon2, comme dans la documentation de FastAPI, ou `bcrypt`). Imposez une longueur minimale.
- **Sessions** : un cookie de session signé (`SessionMiddleware` de Starlette, qui demande `itsdangerous`) ou un jeton JWT dans un cookie `HttpOnly`. La clé secrète vient d'une variable d'environnement, pas du code.
- **Validation** : des modèles Pydantic pour les données reçues. E-mail au bon format et unique, État parmi les 27 États brésiliens, ville reconnue (sinon, message clair).
- **Catalogue** : les produits viennent de vos tables nettoyées, avec le prix que vous avez décidé à l'étape 5 de la demi-journée 1. Pagination obligatoire : il y a plus de 30 000 produits.
- **Panier** : gardé en session tant que la commande n'est pas passée.
- **Requêtes SQL** : toujours paramétrées, jamais construites par concaténation de chaînes. Une connexion (ou un pool de connexions) ouverte au démarrage de l'application, pas à chaque requête.

Indication : avec FastAPI, la vérification des droits s'écrit une fois sous forme de **dépendances** (`Depends`) : par exemple `client_connecte` et `admin_requis`, qui renvoient l'utilisateur courant ou lèvent une `HTTPException` 401 ou 403. Regroupez les routes d'administration dans un `APIRouter` qui déclare `admin_requis` pour toutes ses routes.

Étape 3 : la commande et son fichier CSV

Quand un client valide son panier, la boutique **n'écrit pas la commande dans les tables d'analyse**. Elle crée un nouveau fichier CSV dans un dossier `donnees/entrantes/`, partagé entre les conteneurs. C'est Airflow qui, la séquence suivante, ramassera ces fichiers, les nettoiera et les chargera.

Spécifications du fichier :

- **Un fichier par commande**, nommé avec l'heure UTC et l'identifiant de la commande, par exemple `commande_20261106T101530Z_<order_id>.csv`.
- **Le contenu** : à vous de le définir, mais il doit contenir tout ce qu'il faut pour reconstruire la commande dans votre modèle final (commande, client, articles, paiements), dans un format proche des fichiers Olist pour que vos règles de nettoyage s'appliquent. Un fichier « à plat », une ligne par article avec les colonnes de la commande répétées, est le plus simple. Décrivez le format dans `docs/format_csv_commande.md` : nom, type et sens de chaque colonne.
- **L'écriture atomique** : Airflow ne doit jamais lire un fichier à moitié écrit. Écrivez d'abord sous un nom temporaire, puis renommez (`os.replace`).
- **Les identifiants** : `order_id` au même format que ceux d'Olist (32 caractères hexadécimaux, `uuid.uuid4().hex`).

Dans l'historique du client, une commande dont le fichier n'a pas encore été traité apparaît avec le statut « en cours de traitement ». Comment la boutique le sait-elle ? Notez votre solution dans `CARNET.md`.

Étape 4 : l'espace administrateur et son dashboard

Le dashboard est réservé aux comptes administrateurs. Pour cette séquence, il lit vos tables PostgreSQL (donc les données historiques nettoyées) et affiche au minimum :

1. les 20 dernières commandes ;
2. en haut de page, l'heure à laquelle les données ont été mises à jour pour la dernière fois.

Pour chaque élément, notez dans le carnet la question métier à laquelle il répond et la décision qu'il aide à prendre. Vous reprendrez cette liste la journée 4.

Indication : les requêtes du dashboard portent sur des centaines de milliers de lignes. Si une page met plus d'une seconde à s'afficher, cherchez du côté des index ou d'une table d'agrégats précalculés.

Les indicateurs du dashboard sont calculés à partir de votre modèle en étoile (`fait_ventes` et ses dimensions), pas à partir des tables de la boutique. Comparez avec `EXPLAIN ANALYZE` le temps d'une même requête (chiffre d'affaires par mois et par État) sur les tables nettoyées puis sur le modèle en étoile, et notez les deux temps dans le carnet.

Étape 5 : le robot de trafic et les tests de sécurité

Pour tester votre chaîne de données, il faudra beaucoup de commandes. Écrivez `src/robot_clients.py`, un programme qui se comporte comme des clients : il inscrit des comptes, se connecte, remplit des paniers et passe des commandes **par HTTP, en passant par la boutique** (bibliothèque `requests`, avec une session par client). Options :

- le nombre de clients et le nombre de commandes par minute ;
- des choix réalistes : tirez les villes, les catégories et les moyens de paiement selon leur fréquence dans le jeu Olist ;
- une part de comportements anormaux, pour plus tard : très gros paniers, ville inconnue, paniers abandonnés.

Corrigez ce qui doit l'être et notez les tests dans le carnet.

Demi-journée 3 (5 h) : orchestrer avec Airflow

Objectif : automatiser toute la chaîne. Airflow ramasse les CSV de la boutique, les nettoie avec vos règles de la demi-journée 1, les charge dans PostgreSQL, met à jour les indicateurs et les segments des clients concernés, réentraîne le modèle quand il le faut et surveille la qualité. À la fin, vous mesurez combien de temps une commande met à apparaître sur le dashboard.

Contenu
Étape 1 : Airflow dans le docker-compose, les notions
Étape 2 : le DAG d'ingestion des commandes
Étape 3 : le DAG des indicateurs et des segments
Étape 4 : le DAG de réentraînement
Étape 5 : le DAG de qualité
Étape 6 : mesurer la fraîcheur du dashboard

Étape 1 : Airflow dans le docker-compose

Vous avez déjà vu Ariflow mais si vous avez un doute, cherchez dans la documentation d'Airflow ce que recouvrent ces mots : DAG, tâche, opérateur, dépendance, planification (`schedule`), exécution (`DAG run`), connexion, variable, asset, tâches dynamiques (`expand`). En Airflow 3, on importe `dag`, `task`, `Variable`, `Param` et `Asset` depuis `airflow.sdk`.

Ajoutez un service `airflow` :

- image construite à partir de `apache/airflow:3.x`, avec vos bibliothèques (`pandas`, `scikit-learn`, `joblib`) ;
- commande `standalone`, métadonnées dans une seconde base `airflow` de votre PostgreSQL (pas SQLite), exécuteur `LocalExecutor`, pas de DAG d'exemple ;
- interface sur le port 8081 de votre ordinateur ;
- montages : vos DAG, votre dossier `src` (pour importer `nettoyage.py`), `modeles/`, et `donnees/` en lecture et écriture (Airflow déplacera les fichiers traités) ;
- une connexion `olist_postgres` déclarée par variable d'environnement.

Indications :

- Airflow se règle par variables d'environnement de la forme `AIRFLOW__SECTION__CLE` : cherchez `database.sql_alchemy_conn`, `core.executor`, `core.load_examples`.
- En mode `standalone`, la clé `core.simple_auth_manager_all_admins` à `True` supprime l'écran de connexion. Acceptable en formation, jamais en production.
- Une connexion se déclare avec `AIRFLOW_CONN_OLIST_POSTGRES=postgresql://utilisateur:mot_de_passe@postgres:5432/olist`.
- Pour que vos DAG importent `nettoyage.py`, le dossier `src` doit être dans le `PYTHONPATH` du conteneur.

Étape 2 : le DAG d'ingestion des commandes

Écrivez le DAG `olist_ingestion_commandes`, planifié toutes les 2 minutes, sans rattrapage, une exécution à la fois.

1. Une tâche liste les fichiers en attente dans `donnees/entrantes/` (en ignorant les fichiers temporaires).
2. Une tâche par fichier, créée dynamiquement (`.expand`), qui lit le fichier, vérifie ses colonnes, applique vos fonctions de nettoyage et charge le résultat dans vos tables. Un fichier valide part dans `donnees/traitees/AAAA-MM-JJ/` ; un fichier invalide part dans `donnees/rejetees/` avec un fichier texte qui donne la raison.
3. Une dernière tâche écrit un compte rendu : fichiers traités, rejetés, lignes chargées, durée.

Exigences :

- **Idempotence** : si la même commande est chargée deux fois (un fichier recopié par erreur), la base ne doit pas contenir de doublon. Indication : `INSERT … ON CONFLICT`.
- **Atomicité** : une commande est chargée entièrement (commande, articles, paiements) ou pas du tout.
- **Les mêmes règles** que pour l'historique : aucune règle de nettoyage n'est réécrite dans le DAG.

Lancez le robot à 5 commandes par minute et vérifiez que les fichiers disparaissent de `entrantes/`, que les commandes apparaissent en base et que l'historique du client passe de « en cours de traitement » au vrai statut. Envoyez quelques commandes anormales : sont-elles bien rejetées, avec une raison lisible ?

Questions pour le carnet : que se passe-t-il si Airflow est arrêté une heure ? Si deux exécutions tournaient en même temps ? Si le dossier contient 50 000 fichiers ?

Étape 3 : le DAG des indicateurs et des segments

Quand de nouvelles commandes sont chargées, les indicateurs des clients concernés changent, et parfois leur segment. Écrivez le DAG `olist_indicateurs_segments` :

- il ne tourne pas sur un horaire, mais **quand l'ingestion a mis à jour les commandes**. Indication : déclarez un `Asset` (par exemple `Asset("postgres://olist/commandes")`), mettez-le en sortie (`outlets`) de la tâche de chargement, et utilisez-le comme `schedule` de ce second DAG ;
- il recalcule `indicateurs_clients` uniquement pour les clients touchés depuis le dernier passage (et non pour les 96 000 personnes) ;

- il charge le modèle enregistré et sa standardisation, attribue un segment à chaque client mis à jour (`predict`, sans réentraîner) et met à jour `segments_clients` ;
- il enregistre l'heure de mise à jour affichée en haut du dashboard.

Le même DAG, ou un DAG `olist_etoile` déclenché par le même asset, met aussi à jour le modèle en étoile :

- il ajoute dans `fait_ventes` les seules lignes des nouvelles commandes, sans créer de doublon si le DAG est relancé ;
- il ajoute les nouveaux clients et les nouveaux produits dans leurs dimensions **avant** de charger les faits : pourquoi cet ordre ?
- quand le segment d'un client change, que devient `dim_client` ? Écrasez-vous l'ancien segment, ou gardez-vous l'historique (dimension à évolution lente de type 2, avec des dates de début et de fin de validité) ? Quelle conséquence sur le chiffre d'affaires par segment du mois dernier ?

Dans l'interface, observez le lien entre les deux DAG (vue des assets). Un nouveau client inscrit par le robot apparaît-il sur la carte du dashboard avec un segment ?

Étape 4 : le DAG de réentraînement

Les comportements évoluent : un modèle entraîné sur les données de 2016-2018 vieillit. Écrivez le DAG `olist_reentrainement`, planifié chaque semaine (déclenchez-le à la main pendant le TP) :

1. réentraîne le clustering sur toutes les données, avec le même k ;
2. compare le nouveau modèle à l'ancien : score de silhouette sur un échantillon fixe, taille des segments, part des clients qui changeraient de segment ;
3. **branche** (`@task.branch`) : si le nouveau modèle est meilleur et que moins de 30 % des clients changent de segment, il est publié ; sinon, l'ancien est gardé et une alerte est écrite dans les journaux ;
4. chaque modèle est enregistré avec sa date dans `modeles/`, et une table `historique_modeles` garde ses scores.

Question : pourquoi ne pas simplement réentraîner à chaque nouvelle commande ? Que verrait l'équipe marketing si les numéros de segments changeaient chaque jour ?

Étape 5 : le DAG de qualité

Écrivez le DAG `olist_qualite`, planifié toutes les 10 minutes, qui échoue (rouge) si l'un des contrôles ne passe pas :

- aucune commande sans article, aucun montant négatif, aucun client sans position ;
- le total des paiements de chaque commande récente correspond au total de ses articles, à la tolérance que vous fixez ;
- **fraîcheur** : aucun fichier n'attend dans `entrantes/` depuis plus de 10 minutes ;
- **volume** : le nombre de fichiers rejetés sur la dernière heure reste sous un seuil lu dans une variable Airflow.
- **modèle en étoile** : le chiffre d'affaires de `fait_ventes` est égal à celui des tables nettoyées, et aucune ligne de faits ne pointe vers une dimension absente.

Arrêtez le DAG d'ingestion 15 minutes pendant que le robot tourne : le DAG de qualité doit le détecter.

Étape 6 : mesurer la fraîcheur du dashboard

Avec le robot à 10 commandes par minute, mesurez pour une vingtaine de commandes le temps entre le clic « Commander » et l'apparition de la commande dans les « 20 dernières commandes » du dashboard. Notez la moyenne, le minimum et le maximum dans le carnet, et expliquez d'où vient chaque partie du délai (création du fichier, attente de la planification, démarrage des tâches, nettoyage, chargement).

Reprenez la liste des éléments du dashboard (étape 4 de la demi-journée 2) et, pour chacun, répondez : ce délai est-il acceptable ? Gardez ces réponses : elles ouvrent la journée 4.